COMPUTER ENGINEERING

QUEZON CITY

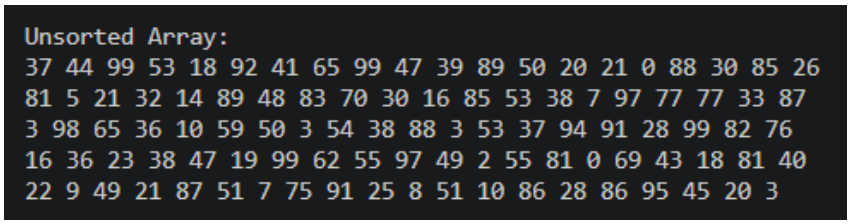
COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Sep 16, 2026	**DATE SUBMITTED:**	Sep 19, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 8.1: Sort Techniques Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE:
Procedure that performed:
ILO A
Task Title & Task Objective
Table 8-1. Array of Values for Sort Algorithm Testing

Screenshot of the code	
Output of the code	
Observation	The main driver code creates an array containing 100 random numbers from 0 to 99. When we print this array at the start, the numbers are totally mixed up and scrambled across the screen. This unorganized list acts as the starting point for all sorting tests, requiring each algorithm to sort the exact same set of data.

```cpp
#include <iostream>
#include "sort.h"
#include <cstdlib> // for generating random integers

const int maxSize = 100;

void displayArr(int arr[])
{
    for (int i = 0; i < maxSize; i++)
    {
        std::cout << arr[i] << " ";
        if ((i + 1) % 20 == 0)
        {
            std::cout << "\n";
        }
    }
    std::cout << "\n";
}

int main()
{
    int unsortedArr[maxSize] = {};
    int shellSortArr[maxSize] = {};
    int mergeSortArr[maxSize] = {};
    int quickSortArr[maxSize] = {};

    // randomly generate an array of integers
    srand(612);
    for (int i = 0; i < maxSize; i++)
    {
        unsortedArr[i] = rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);
```

```
Unsorted Array:
37 44 99 53 18 92 41 65 99 47 39 89 50 20 21 0 88 30 85 26
81 5 21 32 14 89 48 83 70 30 16 85 53 38 7 97 77 77 33 87
3 98 65 36 10 59 50 3 54 38 88 3 53 37 94 91 28 99 82 76
16 36 23 38 47 19 99 62 55 97 49 2 55 81 0 69 43 18 81 40
22 9 49 21 87 51 7 75 91 25 8 51 10 86 28 86 95 45 20 3
```

ILO A.1

Task Title & Task Objective
Table 8-2. Shell Sort Technique

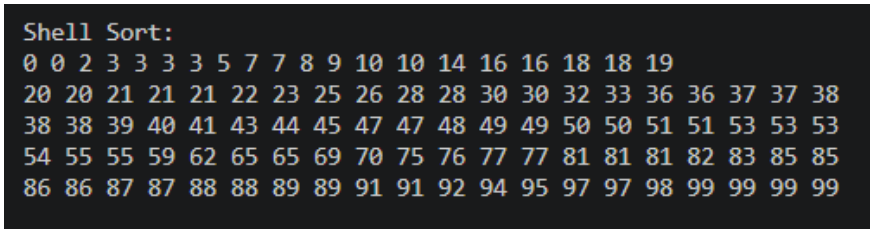
Screenshot of the code

```
    for (int i = 0; i < maxSize; i++)
    {
        shellSortArr[i] = unsortedArr[i];
        mergeSortArr[i] = unsortedArr[i];
        quickSortArr[i] = unsortedArr[i];
    }

    // shell sort
    std::cout << "Shell Sort: \n";
    shellSort(shellSortArr, maxSize);
    displayArr(shellSortArr);
```

Header

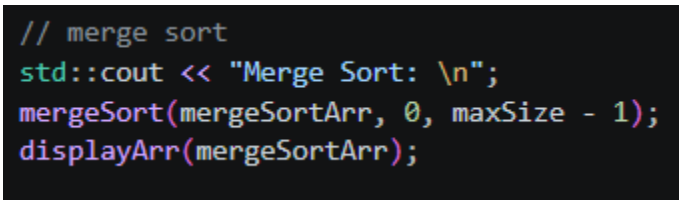
```
1   #ifndef SORT_H
2   #define SORT_H
3
4   template <typename T>
5   void shellSort(T data[], int maxSize)
6   {
7       for (int gap = maxSize / 2; gap > 0; gap /= 2)
8       {
9           for (int i = gap; i < maxSize; i++)
10          {
11              T temp = data[i];
12              int j;
13              for (j = i; j >= gap && data[j - gap] > temp; j -= gap)
14              {
15                  data[j] = data[j - gap];
16              }
17              data[j] = temp;
18          }
19      }
20  }
```

Output of the code	
Observation	The console output for Shell Sort shows that all 100 numbers are completely sorted from the lowest value to the highest value. The algorithm successfully breaks the list into groups using gaps, sorts those groups, and shrinks the gaps until the list is orderly. The last printed array proves that this gap method works correctly to sort the data.

Procedure that performed:

ILO A.2

Task Title & Task Objective
Table 8-3. Merge Sort Algorithm

Screenshot of the code	

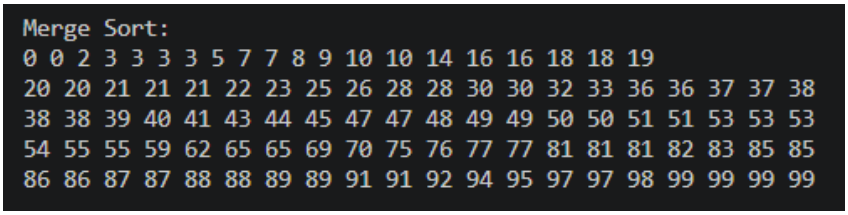
Header with Helper Function

```cpp
// helper function for mergeSort
template <typename T>
void merge(T data[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    T *L = new T[n1];
    T *R = new T[n2];
    for (int i = 0; i < n1; i++)
        L[i] = data[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = data[mid + 1 + j];
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            data[k++] = L[i++];
        else
            data[k++] = R[j++];
    }
    while (i < n1)
        data[k++] = L[i++];
    while (j < n2)
        data[k++] = R[j++];

    delete[] L;
    delete[] R;
}
```

Header

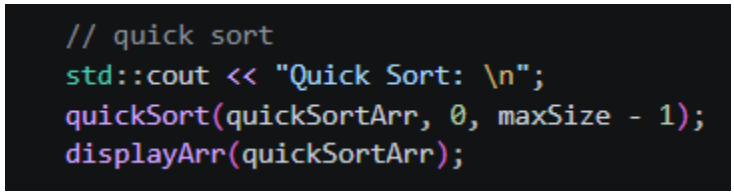
```cpp
template <typename T>
void mergeSort(T data[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;
        mergeSort(data, left, mid);
        mergeSort(data, mid + 1, right);
        merge(data, left, mid, right);
    }
}
```

Output of the code	
Observation	**The console output for Merge Sort displays a perfectly ordered list of numbers with no errors. The algorithm manages this by dividing the 100 numbers into smaller halves, sorting them, and rejoining them using temporary tracking sizes. The clean rows of numbers in the output demonstrate that the splitting and combining strategy works very well.**

Procedure that performed:

ILO A.3

Task Title & Task Objective
Table 8-4. Quick Sort Algorithm

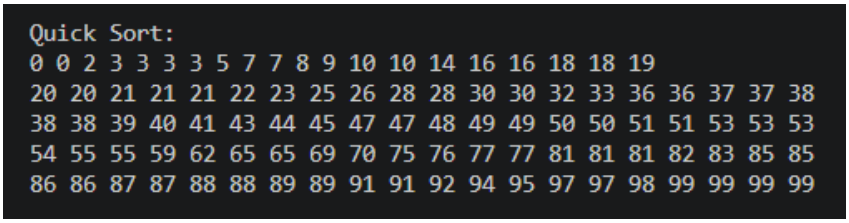
Screenshot of the code	

Header with Helper Function

```cpp
// helper function for quickSort
template <typename T>
int partition(T data[], int left, int right)
{
    T pivot = data[right];
    int i = left - 1;
    for (int j = left; j < right; j++)
    {
        if (data[j] <= pivot)
        {
            i++;
            T temp = data[i];
            data[i] = data[j];
            data[j] = temp;
        }
    }
    T temp = data[i + 1];
    data[i + 1] = data[right];
    data[right] = temp;
    return i + 1;
}
```

Header

```cpp
template <typename T>
void quickSort(T data[], int left, int right)
{
    if (left < right)
    {
        int pivot = partition(data, left, right);
        quickSort(data, left, pivot - 1);
        quickSort(data, pivot + 1, right);
    }
}
#endif
```

Output of the code	
Observation	The console output for Quick Sort shows the exact same sorted list as the other methods. The algorithm handles the 100 numbers by picking a pivot point, moving smaller numbers to the left and bigger numbers to the right, and repeating that process. The final output confirms that the partitioning step correctly puts every single number in its right place.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:

ILO B

Task Title & Objective:

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer:
Yes, it is possible to sort the sublists using a different algorithm after the partitioning step. Instead of using recursion to handle every single small division, it can stop the quick sort algorithm when a sublist becomes small and switch to an iterative algorithm like Insertion Sort to finish the job.

If we Imagine we have an unsorted array of 8 elements: [34, 12, 54, 87, 19, 43, 30, 25]. If we choose the [54] as our pivot, the partitioning step will rearrange the items into two sections. The left section contains numbers smaller than the pivot: [34, 12, 19, 43, 30, 25]. The right section contains numbers larger than the pivot, which is the [87]. Instead of running Quick Sort on these two groups, we can pass them directly to Insertion Sort. Insertion Sort rearranges the left group to [12, 19, 25, 30, 34, 43] and the right group stays as [87]. When you join them back with the pivot, you get the final sorted array: [12, 19, 25, 30, 34, 43, 54, 87].

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N * log N) for their time complexity?

For a small list containing only 15 numbers, Insertion Sort gives the fastest speed. This happens because basic sorting routines do not require complex background setup code or extra memory tracking, making them very quick for small sets of data. Out of the advanced algorithms, Merge Sort offers the most reliable speed because its processing time stays exactly the same even if the numbers are completely scrambled.

Why can merge sort and quick sort have O(N • log N) for their time complexity?

Merge Sort and Quick Sort reach an O(NlogN) time complexity because they use a divide and conquer strategy to process lists. The log N part represents the total number of times the array is split in half until only single numbers are left. The N part represents the actual work done to compare, partition, and merge those items back together at each layer. Multiplying the number of split levels by the sorting work done on each level results in a total complexity of O(NlogN).

Summary of lessons learned	This activity taught me that advanced sorting methods are much better at handling larger sets of data than previous sorting functions we used. I discovered that dividing a large problem into smaller parts makes programs run much faster.
Analysis of the procedure	The procedures of each ILO are performed well to test the three advanced sorting algorithms under identical conditions. First, a single array of 100 random integers was created, and copies of this array were distributed to each sorting function to ensure a completely fair test. The procedure tracked how different strategies organize data, showing that Shell Sort groups elements using decreasing gap intervals, Merge Sort splits arrays evenly into smaller pieces to merge them back, and Quick Sort uses a partition step around a pivot point. The final console outputs showed that every single method successfully turned the messy list into the exact same sorted array layout.
Analysis of the supplementary	For a small list of exactly 15 numbers, Insertion Sort is the fastest because simple sorting routines do not require complex background setup code or extra memory tracking. It is also possible to use a different sorting algorithm on the sublists after the partitioning step is completed, allowing the quick sort recursion to switch to a simpler method such as Insertion Sort when a sublist becomes small. Merge Sort and Quick Sort have O(NlogN) time complexity because they use a divide and conquer method to process lists. The log N part represents the total number of times the array is split in half until only single numbers are left, while the N part represents the actual work done to compare and merge or partition those items back together at each layer.

Conclusion

This activity successfully demonstrated the group of sorting algorithms that use divide and conquer methods and recursive techniques to reach an O(nlogn) time complexity. This approach is useful for managing larger datasets, such as the 100 element array of random values tested during the laboratory procedures. The algorithms examined include shell sort, merge sort, and quick sort, which were all applied to organize unsorted data using a fixed random seed value to ensure consistent, replicable results. While merge sort splits the main array directly into equal halves, quick sort relies on a pivot element to partition the numbers into sublists. The activity also showed that these sublists can be sorted using other methods like insertion sort to optimize the overall process.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.